

第13章 GPIO 输入—按键检测

按键检测使用到 GPIO 外设的基本输入功能，本章中不再赘述 GPIO 外设的概念，如您忘记了，可重读前面“使用寄存器点亮 LED 灯—GPIO 框图剖析”小节，STM32 标准库中 GPIO 初始化结构体 GPIO_TypeDef 的定义与“自己写库—构建库函数雏形的定义引脚模式的枚举类型”小节中讲解的相同。

13.1 硬件设计

按键机械触点断开、闭合时，由于触点的弹性作用，按键开关不会马上稳定接通或一下子断开，使用按键时会产生图 13-1 中的带波纹信号，需要用软件消抖处理滤波，不方便输入检测。本实验板连接的按键带硬件消抖功能，见图 13-2，它利用电容充放电的延时，消除了波纹，从而简化软件的处理，软件只需要直接检测引脚的电平即可。

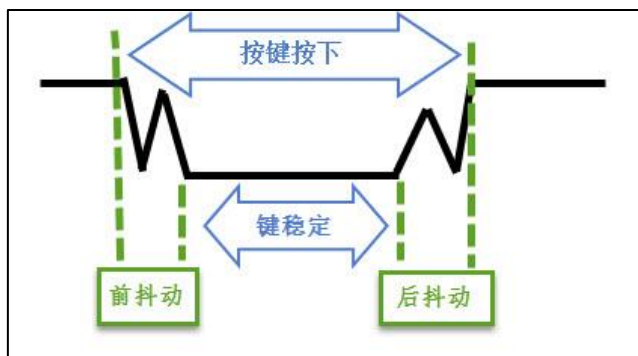


图 13-1 按键抖动说明图

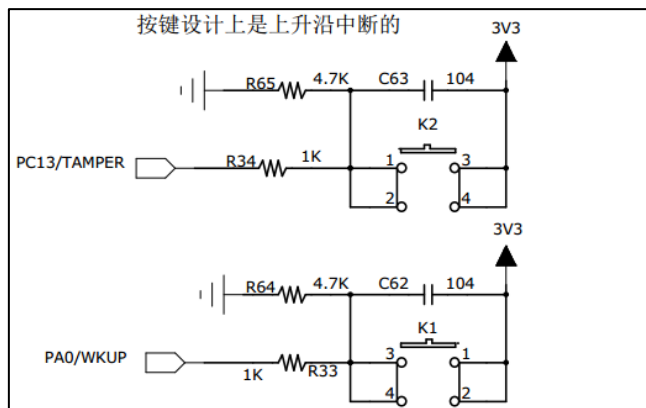


图 13-2 按键原理图

从按键的原理图可知，这些按键在没有被按下的时候，GPIO 引脚的输入状态为低电平(按键所在的电路不通，引脚接地)，当按键按下时，GPIO 引脚的输入状态为高电平(按键所在的电路导通，引脚接到电源)。只要我们检测引脚的输入电平，即可判断按键是否被按下。

若您使用的实验板按键的连接方式或引脚不一样，只需根据我们的工程修改引脚即可，程序的控制原理相同。

13.2 软件设计

同 LED 的工程，为了使工程更加有条理，我们把按键相关的代码独立分开存储，方便以后移植。在“工程模板”之上新建“bsp_key.c”及“bsp_key.h”文件，这些文件也可根据您的喜好命名，这些文件不属于 STM32 标准库的内容，是由我们自己根据应用需要编写的。

13.2.1 编程要点

1. 使能 GPIO 端口时钟；
2. 初始化 GPIO 目标引脚为输入模式(引脚默认电平受按键电路影响，浮空/上拉/下拉均没有区别)；
3. 编写简单测试程序，检测按键的状态，实现按键控制 LED 灯。

13.2.2 代码分析

1. 按键引脚宏定义

同样，在编写按键驱动时，也要考虑更改硬件环境的情况。我们把按键检测引脚相关的宏定义到“bsp_key.h”文件中，具体见代码清单 12-1。

代码清单 13-1 按键检测引脚相关的宏

```
1 //引脚定义
2 /*****
3 #define KEY1_PIN                GPIO_Pin_0
4 #define KEY1_GPIO_PORT          GPIOA
5 #define KEY1_GPIO_CLK            RCC_AHB1Periph_GPIOA
6
7 #define KEY2_PIN                GPIO_Pin_13
8 #define KEY2_GPIO_PORT          GPIOC
9 #define KEY2_GPIO_CLK            RCC_AHB1Periph_GPIOC
10 *****/
11
12 /** 按键按下标置宏
13 * 按键按下为高电平，设置 KEY_ON=1, KEY_OFF=0
14 * 若按键按下为低电平，把宏设置成 KEY_ON=0, KEY_OFF=1 即可
15 */
16 #define KEY_ON 1
17 #define KEY_OFF 0
```

以上代码根据按键的硬件连接，把检测按键输入的 GPIO 端口、GPIO 引脚号以及 GPIO 端口时钟封装起来了。

2. 按键 GPIO 初始化函数

利用上面的宏，编写按键的初始化函数，具体见代码清单 13-2。

代码清单 13-2 按键 GPIO 初始化函数

```
1 /**
2 * @brief 配置按键用到的 I/O 口
3 * @param 无
4 * @retval 无
5 */
```

```
6 void Key_GPIO_Config(void)
7 {
8     GPIO_InitTypeDef GPIO_InitStructure;
9
10    /*开启按键 GPIO 口的时钟*/
11    RCC_AHB1PeriphClockCmd(KEY1_GPIO_CLK|KEY2_GPIO_CLK,ENABLE);
12
13    /*选择按键的引脚*/
14    GPIO_InitStructure.GPIO_Pin = KEY1_PIN;
15
16    /*设置引脚为输入模式*/
17    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN;
18
19    /*设置引脚不上拉也不下拉*/
20    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;
21
22    /*使用上面的结构体初始化按键*/
23    GPIO_Init(KEY1_GPIO_PORT, &GPIO_InitStructure);
24
25    /*选择按键的引脚*/
26    GPIO_InitStructure.GPIO_Pin = KEY2_PIN;
27
28    /*使用上面的结构体初始化按键*/
29    GPIO_Init(KEY2_GPIO_PORT, &GPIO_InitStructure);
30 }
```

同为 GPIO 的初始化函数，初始化的流程与“LED GPIO 初始化函数”章节中的类似，主要区别是引脚的模式。函数执行流程如下：

- (1) 使用 GPIO_InitTypeDef 定义 GPIO 初始化结构体变量，以便下面用于存储 GPIO 配置。
- (2) 调用库函数 RCC_AHB1PeriphClockCmd 来使能按键的 GPIO 端口时钟，调用时我们使用“|”操作同时配置两个按键的时钟。
- (3) 向 GPIO 初始化结构体赋值，把引脚初始化成浮空输入模式，其中的 GPIO_Pin 使用宏“KEYx_PIN”来赋值，使函数的实现方便移植。由于引脚的默认电平受按键电路影响，所以设置成“浮空/上拉/下拉”模式均没有区别。
- (4) 使用以上初始化结构体的配置，调用 GPIO_Init 函数向寄存器写入参数，完成 GPIO 的初始化，这里的 GPIO 端口使用“KEYx_GPIO_PORT”宏来赋值，也是为了程序移植方便。
- (5) 使用同样的初始化结构体，只修改控制的引脚和端口，初始化其它按键检测时使用的 GPIO 引脚。

3. 检测按键的状态

初始化按键后，就可以通过检测对应引脚的电平来判断按键状态了，具体见代码清单 13-3。

代码清单 13-3 检测按键的状态

```
1 /**
2  * @brief 检测是否有按键按下
3  * @param GPIOx:具体的端口, x 可以是 (A...K)
4  * @param GPIO_PIN:具体的端口位, 可以是 GPIO_PIN_x (x 可以是 0...15)
5  * @retval 按键的状态
6  * @arg KEY_ON:按键按下
7  * @arg KEY_OFF:按键没按下
8  */
9 uint8_t Key_Scan(GPIO_TypeDef* GPIOx,uint16_t GPIO_Pin)
10 {
11     /*检测是否有按键按下 */
12     if (GPIO_ReadInputDataBit(GPIOx,GPIO_Pin) == KEY_ON ) {
13         /*等待按键释放 */
14         while (GPIO_ReadInputDataBit(GPIOx,GPIO_Pin) == KEY_ON);
15         return KEY_ON;
16     } else
17         return KEY_OFF;
18 }
```

在这里我们定义了一个 Key_Scan 函数用于扫描按键状态。GPIO 引脚的输入电平可通过读取 IDR 寄存器对应的数据位来感知, 而 STM32 标准库提供了库函数 GPIO_ReadInputDataBit 来获取位状态, 该函数输入 GPIO 端口及引脚号, 函数返回该引脚的电平状态, 高电平返回 1, 低电平返回 0。Key_Scan 函数中以 GPIO_ReadInputDataBit 的返回值与自定义的宏“KEY_ON”对比, 若检测到按键按下, 则使用 while 循环持续检测按键状态, 直到按键释放, 按键释放后 Key_Scan 函数返回一个“KEY_ON”值; 若没有检测到按键按下, 则函数直接返回“KEY_OFF”。若按键的硬件没有做消抖处理, 需要在这个 Key_Scan 函数中做软件滤波, 防止波纹抖动引起误触发。

4. 主函数

接下来我们使用主函数编写按键检测流程, 见代码清单 13-4。

代码清单 13-4 按键检测主函数

```
1 #include "stm32f4xx.h"
2 #include "../led/bsp_led.h"
3 #include "../key/bsp_key.h"
4
5 /**
6  * @brief 主函数
7  * @param 无
8  * @retval 无
9  */
10 int main(void)
11 {
12     /* LED 端口初始化 */
13     LED_GPIO_Config();
14
15     /*初始化按键*/
16     Key_GPIO_Config();
17
18
19     /* 轮询按键状态, 若按键按下则反转 LED */
20     while (1) {
21         if ( Key_Scan(KEY1_GPIO_PORT,KEY1_PIN) == KEY_ON ) {
22             /*LED1 反转*/
23             LED1_TOGGLE;
24         }
25
26         if ( Key_Scan(KEY2_GPIO_PORT,KEY2_PIN) == KEY_ON ) {
```

```
27         /*LED2 反转*/
28         LED2_TOGGLE;
29     }
30 }
31 }
```

代码中初始化 LED 灯及按键后，在 while 函数里不断调用 Key_Scan 函数，并判断其返回值，若返回值表示按键按下，则反转 LED 灯的状态。

13.3 下载验证

把编译好的程序下载到开发板并复位，按下按键可以控制 LED 灯亮、灭状态。